Tech03 Agendamento Hospitalar

LINK PARA ACESSO AO PROJETO: https://github.com/MichelleAyumi/FIAP

1. Visão geral

O Tech03 é um backend para agendamento hospitalar, consulta de histórico e envio assíncrono de confirmações e lembretes. A API GraphQL aplica permissões por perfil autenticado, enquanto o serviço de notificações recebe eventos via RabbitMQ e agenda emails. A solução utiliza persistência separada por serviço, controles de acesso, recuperação de falhas de mensageria e testes de integração.

Escopo: API sem interface clínica web, sem cadastro de usuários e sem cancelamento de consultas. O perfil demo cria usuários fictícios

2. Requisitos e regras de negócio

Operação	Médico	Enfermeiro	Paciente
Consultar histórico/consulta	Permitido	Permitido	Somente próprios
Criar/reagendar consulta	Permitido	Permitido	Negado
Editar registro clínico	Permitido	Negado	Negado

O perfil MONITOR acessa métricas, não a API clínica. Para pacientes, a autorização também verifica a propriedade do registro.

Validações principais: data futura com fuso horário; duração entre 10 e 240 minutos; motivo obrigatório; paciente e médico existentes; bloqueio de sobreposição de horários; controle de concorrência por expectedVersion; clinicalNotes editável apenas por médicos. Consultas passadas não podem ser reagendadas.

3. Arquitetura e responsabilidades

Módulo	Responsabilidade
contracts	Contrato AppointmentEvent e topologia RabbitMQ compartilhada.
scheduling-service	GraphQL, autenticação/autorização, regras de agenda, JDBC e outbox.
notification-service	Consumo de eventos, inbox, lembretes e envio SMTP.
infra	Prometheus, alertas e dashboard Grafana.
postman / scripts / docs	Testes de API, smoke test, demonstração e evidências.

4. Persistência e mensageria

O serviço de agendamento mantém usuários, pacientes, médicos, consultas e outbox_events. O serviço de notificações mantém inbox_events, snapshots das consultas, notification_jobs e coordenação transacional. No Docker, cada serviço possui PostgreSQL próprio; localmente usa H2 separado e os testes usam H2 em memória. O schema é inicializado por schema.sql, sem migrações versionadas.

Fluxo assíncrono:

1) GraphQL autentica e grava consulta + evento na mesma transação;

2) OutboxPublisher publica no RabbitMQ e confirma o envio;

3) notificações deduplicam o evento e agendam jobs;

4) ReminderDispatcher envia os emails por SMTP. A confirmação é imediata e o lembrete é programado para 24h antes.

A outbox retém eventos quando o broker falha; a inbox deduplica eventId e ignora versões antigas. Falhas SMTP recebem até cinco tentativas. Não há garantia externa de exactly-once: uma queda entre envio e persistência pode repetir um email.

5. Contrato GraphQL

Endpoint: POST http://localhost:8080/graphql, com HTTP Basic e Content-Type application/json. Operações principais: createAppointment, updateAppointment, updateClinicalNotes, appointment e history. Na edição, expectedVersion deve corresponder à versão mais recente.

```
mutation Criar($input: AppointmentInput!) {
  createAppointment(input: $input) { id patientId doctorId startsAt version }
}
query {
  history(patientId: "pac-001", futureOnly: true, limit: 20, offset: 0)
```

```
  { id startsAt version }
}
```

Credenciais inválidas retornam HTTP 401. Erros GraphQL podem retornar HTTP 200 com errors e classificações como FORBIDDEN, BAD_REQUEST ou NOT_FOUND; o cliente deve verificar também o campo errors.

6. Execução local

Na pasta Tech03:

```
Copy-Item .env.example .env
docker compose --profile monitoring up -d --build
docker compose ps
.\mvnw.cmd -B -ntp verify
python scripts/smoke_test.py
```

Recurso	Endereço local
GraphQL	localhost:8080/graphql
Mailpit	localhost:8025
RabbitMQ	localhost:15672
Grafana	localhost:3000
Prometheus	localhost:9090

7. Testes e observabilidade

Verificação	Resultado registrado
Integração de agendamento	10 testes aprovados
Integração de notificações	7 testes aprovados
Coleção Newman	10 requisições / 19 verificações aprovadas
Ponta a ponta	Criação, email, atualização e DLQ verificados
Recuperação	Outbox após queda do broker e lembretes verificados

Actuator expõe saúde e métricas; Prometheus coleta os serviços e Grafana apresenta painéis de disponibilidade, outbox, latência, requisições, lembretes, heap e falhas. Há alertas para indisponibilidade, outbox pendente e jobs com falha. Os logs são estruturados em JSON; não há tracing distribuído.